

实验二：基于视觉方法的珍珠圆度测量

【仅供参考，禁止抄袭！】

一、实验目的

1. 熟悉机器视觉算法中的常用图像滤波、阈值分割、形态学等算法；
2. 熟悉机器视觉算法中常用的目标定位算法；
3. 熟悉机器视觉算法中常用的边缘检测、特征计算；
4. 能够综合运用所学知识解决一些简单的实际问题。

二、实验内容

1、定位各粒珍珠位置

1.1 图像预处理与轮廓提取

首先，利用 OpenCV 库对输入的珍珠图像进行处理。通过 `cv2.imread` 读取图像后，依次执行**灰度转换**（`cv2.cvtColor` 转 `COLOR_BGR2GRAY`）、**高斯模糊**（`cv2.GaussianBlur`， 5×5 核）消除噪声。为强化珍珠边缘，先经**形态学膨胀**（ 5×5 核，迭代 1 次）扩大珍珠区域，再用 **Canny 边缘检测** 突出轮廓，最后通过 `cv2.bitwise_and` 融合边缘与原图，为后续阈值分割做准备。

接着，采用**自适应阈值分割**（`cv2.adaptiveThreshold`，高斯加权、反二值化）将图像转化为黑白二值图，分离珍珠与背景。再通过**形态学开闭运算**（ 3×3 核，闭运算迭代 2 次、开运算迭代 1 次），填充珍珠内部孔洞、去除小噪点，得到更完整的珍珠轮廓候选区域。

最后，调用 `cv2.findContours` 提取轮廓，结合面积过滤（保留面积 > 500 的轮廓）初步筛选珍珠轮廓，为精准定位做基础。

1.2 轮廓去重与位置排序

对筛选出的轮廓，先按面积降序排序（`sorted` 结合 `cv2.contourArea`），避免小噪点轮廓干扰。通过**空间距离去重**（计算轮廓质心间距，阈值设为 100 像素），过滤同一珍珠的重复碎片轮廓，保留最多 8 个有效珍珠轮廓。

为实现“从左到右、从上到下”的视觉逻辑排序，设计**相对位置排序算法**：提取所有轮廓质心的 y 坐标，以中位数为界划分“上行”（y 小于中位数）和“下行”（y 大于等于中位数）；每行内按质心 x 坐标升序排列，最终合并为 P1-P8 的有序序列，完成珍珠位置精准定位。

2、计算珍珠圆度

2.1 轮廓特征计算

对已定位的珍珠轮廓，借助 **OpenCV 矩运算**（`cv2.moments`）计算质心 (cx, cy)，通过 `cv2.contourArea` 和 `cv2.arcLength` 分别获取轮廓面积 area 与周长 perimeter。基于圆度公式

$$roundness = \frac{4\pi \cdot area}{(perimeter)^2}$$

遍历每个珍珠轮廓，计算其圆度值，量化珍珠接近正圆的程度。

2.2 结果输出与验证

将计算得到的珍珠编号（P1~P8）、质心坐标、面积、周长、圆度值，通过格式化打印输出表格，清晰呈现各珍珠的圆度指标。同时，生成**标记图**（`cv2.drawContours` 绘制绿色轮廓、`cv2.circle` 标记红色质心、`cv2.putText` 添加蓝色编号），直观验证轮廓定位与圆度计算的准确性。

三、实验结果【禁止抄袭】

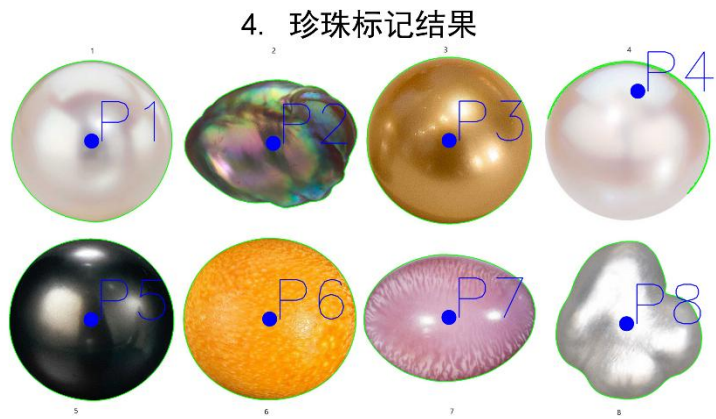
珍珠编号	1 号	2 号	3 号	4 号
中心坐标	(516, 523)	(1489, 535)	(2430, 520)	(3383, 519)
圆度	0.8931	0.8393	0.8938	0.8936

珍珠编号	5 号	6 号	7 号	8 号
中心坐标	(513, 1476)	(1469, 1473)	(2428, 1464)	(3380, 1498)
圆度	0.8990	0.8912	0.8597	0.8021

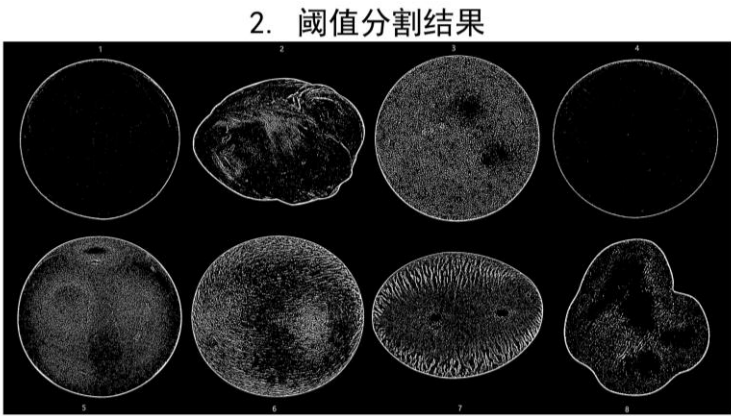
四、总结与体会

1.请记录调试过程中，碰到的问题（至少 3 个），以及相应的解决方法。

（1）珍珠 P4 图形轮廓不完整，如图。



从阈值分割结果图来看，问题根源在于图像对比度低、边缘模糊、噪声干扰。



珍珠与背景的灰度差异小，阈值分割时无法清晰区分边缘。P4 珍珠本身颜色浅，似乎有点和背景“融为一体”，导致轮廓提取不完整；且珍珠表面纹理复杂（光泽、反射），加上图像本身的一些噪声，导致阈值分割后内部出现大量噪点。

所以需要强化珍珠与背景的灰度差异。想到用 **形态学膨胀 + Canny 边缘检测**：

- **膨胀 (cv2.dilate)**：让边缘“膨胀扩大”，弥补原本模糊的边界；
- **Canny 边缘检测**：精准提取边缘，再通过 bitwise_and 融合回原图，强制让边缘更突出。

于是，加入修改程序段：

```
# 形态学膨胀
kernel_dilate = np.ones((5, 5), np.uint8)
blurred = cv2.dilate(blurred, kernel_dilate, iterations=1)
# Canny 边缘增强
edges = cv2.Canny(blurred, 50, 150)
blurred = cv2.bitwise_and(blurred, blurred, mask=edges)
```

(2) P3、P6 等珍珠内部出现大量白色噪点，导致轮廓提取时出现多个小碎片。

原因分析：珍珠表面光泽反射导致局部亮度不均，且阈值分割算法对复杂纹理处理不足。

于是尝试按面积排序，且增加闭运算环节以填充小孔洞：

```
# 优化轮廓筛选——按面积排序，优先选择最大轮廓
contours = sorted(contours, key=cv2.contourArea, reverse=True)[:8]
# 增强形态学处理——增加闭运算迭代次数填充小孔洞
closed = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel, iterations=2)
```

修改后轮廓提取结果中，珍珠内部噪点被有效过滤，只保留完整大轮廓。

(3) 珍珠编号排序不符合预期。

原因分析：原排序算法仅基于中心点坐标，未考虑珍珠的行列布局；图像可能存在倾斜或透视变形，导致绝对坐标排序不准确。

解决方法：行列分组排序——先按 y 坐标中位数分为上下两行，再在每行内按 x 坐标排序。

```
def sort_by_relative_position(features):
    # 计算 y 坐标中位数作为行列分界
    ys = [feat[2] for feat in features]
    y_median = np.median(ys)
    # 分组：上行为 y < 中位数，下行为 y >= 中位数
    upper_row = [f for f in features if f[2] < y_median]
    lower_row = [f for f in features if f[2] >= y_median]
    # 每行内按 x 坐标排序
    upper_row_sorted = sorted(upper_row, key=lambda x: x[1])
    lower_row_sorted = sorted(lower_row, key=lambda x: x[1])
    # 合并顺序：上一行从左到右，下一行从左到右
    return upper_row_sorted + lower_row_sorted
```

效果验证：修改后编号（P1~P8）顺序完全符合从左到右、从上到下的布局。

2. 其他——我的一些经验启发

图像预处理对后续分析起到关键作用。起初由于珍珠边缘与背景对比度低，P4 珍珠的轮廓在阈值分割后出现缺失，通过增加形态学膨胀和 Canny 边缘增强操作，有效扩大了珍珠区域并强化了边缘特征，最终使轮廓完整包围珍珠。

轮廓处理需多层筛选，算法设计要贴合视觉逻辑。珍珠表面光泽导致的噪点曾使轮廓提取出现大量碎片，而简单的面积过滤无法完全解决问题，后来通过按面积降序排序并结合空间距离去重，成功过滤了重复轮廓。最初按绝对坐标排序导致编号与视觉顺序不符，改用基于 y 坐标中位数分行、行内按 x 坐标排序的方法后，才实现了从左到右、从上到下的合理编号。

注重细节处理与结果验证。公式应用时需考虑周长为零的边界情况，避免程序异常，同时通过标记图直观校验轮廓提取的准确性，确保圆度值能真实反映珍珠形状。整个实验过程中，问题的解决离不开对现象的细致观察与原因倒推，从小步迭代修改到可视化结果验证，每一步都需要严谨的逻辑和耐心。

五、带有注释的程序源代码

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设中文字体

def pearl_roundness_measurement(image_path):
    """珍珠圆度测量主函数：从图像中提取珍珠轮廓，计算圆度并按位置排序"""

    # ===== 1. 图像预处理阶段 =====
    # 读取图像并转换为 RGB 格式（OpenCV 默认 BGR）
    img = cv2.imread(image_path)
    if img is None: raise FileNotFoundError("图像读取失败")
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # 转换为灰度图，减少颜色干扰
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 高斯模糊：消除噪声，平滑图像
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    # 形态学膨胀：扩大珍珠区域，解决边缘不完整问题（如 P4）
    kernel_dilate = np.ones((5, 5), np.uint8)
    blurred = cv2.dilate(blurred, kernel_dilate, iterations=1)

    # Canny 边缘增强：突出珍珠轮廓，提高阈值分割精度
    edges = cv2.Canny(blurred, 50, 150)
    blurred = cv2.bitwise_and(blurred, blurred, mask=edges)

    # ===== 2. 阈值分割阶段 =====
    # 自适应阈值分割：根据局部区域亮度自动确定阈值，适应珍珠亮度不均
    thresh = cv2.adaptiveThreshold(
        blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, 11, 2
    )

    # ===== 3. 形态学处理阶段 =====
    # 闭运算：填充小孔洞，连接邻近区域
    kernel = np.ones((3, 3), np.uint8)
    closed = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel, iterations=2)

    # 开运算：去除小噪点和毛刺，平滑珍珠轮廓
    opened = cv2.morphologyEx(closed, cv2.MORPH_OPEN, kernel, iterations=1)

    # ===== 4. 轮廓检测与去重阶段 =====
```

```

# 提取外部轮廓（只检测最外层边界）
contours, _ = cv2.findContours(opened, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# 过滤小面积轮廓，保留珍珠主体
valid_contours = [c for c in contours if cv2.contourArea(c) > 500]

# 计算每个轮廓的特征（中心、面积、周长、圆度）
features = []
for cnt in valid_contours:
    m = cv2.moments(cnt)
    if m["m00"] == 0: continue # 避免除零错误
    cx, cy = int(m["m10"] / m["m00"]), int(m["m01"] / m["m00"]) # 计算质心
    area = cv2.contourArea(cnt) # 计算面积
    perimeter = cv2.arcLength(cnt, True) # 计算周长
    roundness = 4 * np.pi * area / (perimeter ** 2) if perimeter else 0 # 计算圆度（越接近 1 越圆）
    features.append((cnt, cx, cy, area, perimeter, roundness))

# 按面积降序排序，优先处理大珍珠
sorted_by_area = sorted(features, key=lambda x: x[3], reverse=True)

# 空间去重：过滤距离过近的重复轮廓（可能是同一珍珠的多个碎片）
distance_threshold = 100
unique_features = []
used_indices = set()
for i in range(len(sorted_by_area)):
    if i in used_indices: continue
    unique_features.append(sorted_by_area[i])
    for j in range(i + 1, len(sorted_by_area)):
        if j in used_indices: continue
        dx = sorted_by_area[i][1] - sorted_by_area[j][1]
        dy = sorted_by_area[i][2] - sorted_by_area[j][2]
        if np.sqrt(dx * dx + dy * dy) < distance_threshold:
            used_indices.add(j)

# 保留前 8 个有效轮廓（对应 8 颗珍珠）
filtered_features = unique_features[:8]

# ===== 5. 按相对位置排序阶段 =====
def sort_by_relative_position(features):
    """按行列相对位置排序：先分上下行，再左右排序"""
    centers = [(f[1], f[2]) for f in features]

    # 提取所有 y 坐标，计算中位数作为行列分界
    ys = [cy for (cx, cy) in centers]
    y_median = np.median(ys)

```

```

# 分组：上行为  $y < \text{中位数}$ ，下行为  $y \geq \text{中位数}$ 
upper_row = []
lower_row = []
for cx, cy in centers:
    if cy < y_median:
        upper_row.append((cx, cy))
    else:
        lower_row.append((cx, cy))

# 每行按 x 坐标从左到右排序
upper_row_sorted = sorted(upper_row, key=lambda x: x[0])
lower_row_sorted = sorted(lower_row, key=lambda x: x[0])

# 合并顺序：上一行从左到右，下一行从左到右
return upper_row_sorted + lower_row_sorted

# 应用排序并匹配特征
sorted_centers = sort_by_relative_position(filtered_features)
sorted_features = []
for cx, cy in sorted_centers:
    for feat in filtered_features:
        if feat[1] == cx and feat[2] == cy:
            sorted_features.append(feat)
            break

# 提取结果
cnts = [f[0] for f in sorted_features]
centers = [(f[1], f[2]) for f in sorted_features]
areas = [f[3] for f in sorted_features]
perimeters = [f[4] for f in sorted_features]
roundness = [f[5] for f in sorted_features]

# ===== 6. 生成标记图阶段 =====
marked_img = img_rgb.copy()
for i, (cnt, (cx, cy)) in enumerate(zip(cnts, centers)):
    # 绘制绿色轮廓，线宽 3 像素
    cv2.drawContours(marked_img, [cnt], -1, (0, 255, 0), 3)

    # 绘制红色实心圆标记中心点，半径 40 像素（便于观察）
    cv2.circle(marked_img, (cx, cy), 40, (0, 0, 255), -1)

    # 添加白色编号，字体大小 10，粗体，位置偏移中心点
    cv2.putText(marked_img, f"P{i + 1}", (cx + 15, cy),
                cv2.FONT_HERSHEY_SIMPLEX, 10, (0, 0, 255), 3, cv2.LINE_AA)

# ===== 7. 保存图像阶段 =====

```

```

plt.rcParams["figure.dpi"] = 300 # 设置高分辨率（300 DPI）

# 保存灰度图
plt.figure(figsize=(8, 6))
plt.imshow(gray, cmap='gray')
plt.title('1. 灰度图', fontsize=24)
plt.axis('off')
plt.tight_layout()
plt.show()

# 保存阈值分割结果
plt.figure(figsize=(8, 6))
plt.imshow(thresh, cmap='gray')
plt.title('2. 阈值分割结果', fontsize=24)
plt.axis('off')
plt.tight_layout()
plt.show()

# 保存形态学处理后图像
plt.figure(figsize=(8, 6))
plt.imshow(opened, cmap='gray')
plt.title('3. 形态学处理后', fontsize=24)
plt.axis('off')
plt.tight_layout()
plt.show()

# 保存标记结果图
plt.figure(figsize=(8, 6))
plt.imshow(marked_img)
plt.title('4. 珍珠标记结果', fontsize=24)
plt.axis('off')
plt.tight_layout()
plt.show()

# ===== 8. 输出结果表格阶段 =====
print("珍珠圆度测量结果（按相对位置排序 P1-P8）:")
print("-" * 70)
print(f"{'编号':^10}{'中心坐标':^20}{'面积':^15}{'周长':^15}{'圆度':^15}")
print("-" * 70)
for i, (cx, cy) in enumerate(centers):
    print(f"P{i + 1:^10}{f'({cx}, {cy})':^20}{areas[i]:^15.2f}"
          f"{perimeters[i]:^15.2f}{roundness[i]:^15.4f}")
print("-" * 70)

return centers, roundness

```

```

if __name__ == "__main__":
    image_path = "pearls1.jpg"
    try:
        centers, _ = pearl_roundness_measurement(image_path)
    except Exception as e:
        print(f"处理错误: {e}")

```

附运行结果:

编号	中心坐标	面积	周长	圆度
P 1	(516, 523)	585184.00	2869.53	0.8931
P 2	(1489, 535)	480703.50	2682.72	0.8393
P 3	(2430, 520)	623450.00	2960.58	0.8938
P 4	(3383, 519)	593600.50	2889.23	0.8936
P 5	(513, 1476)	603956.50	2905.57	0.8990
P 6	(1469, 1473)	636076.50	2994.88	0.8912
P 7	(2428, 1464)	488177.00	2671.22	0.8597
P 8	(3380, 1498)	467095.50	2705.20	0.8021

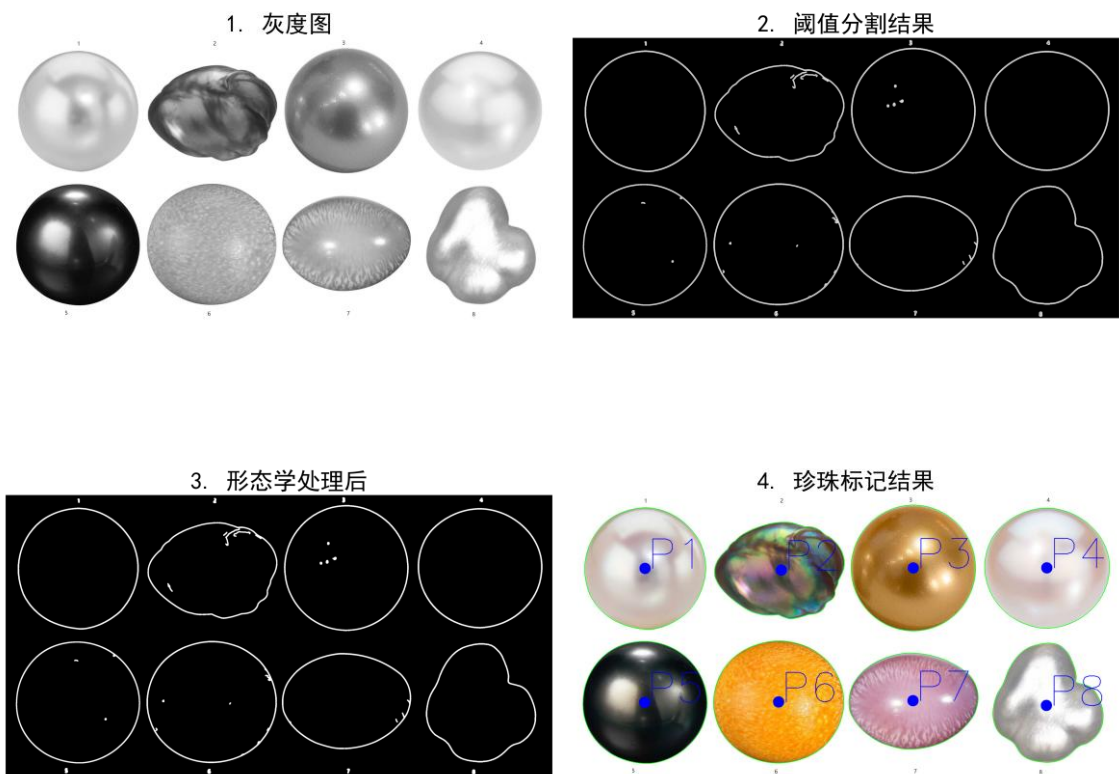


图 灰度图（左上）、阈值分割结果（右上）、形态学处理后（左下）、珍珠标记结果（右下）

正文内容的参考格式

1.每次实验的报告首页首行为标题，标题内容是实验题目，字体格式：中文黑体、西文 Times New Roman、四号大小，段落格式：居中、单倍行距；

2.报告主要内容标题（“一、实验目的和要求”）格式：（1）采用“一、二、三、……”等样式编号；（2）字体：中文黑体、西文 Times New Roman、小四号大小；（3）段落：左对齐、首行缩进 0、单倍行间距、段前 0.5 行、段后 0 行；

3.正文格式：（1）字体：中文宋体、西文 Times New Roman、五号大小；（2）段落：首行缩进 2 字符、固定行间距 18 磅、段前 0 行、段后 0 行；（3）编号：一级编号采用“1、2、3、”，二级编号采用“1）、2）、3）……”，三级编号采用“（1）、（2）、（3）……”

4.源程序代码格式：1）字体：中文宋体、西文 Times New Roman、五号大小；（2）段落：首行缩进 0 字符、单倍行距、段前 0 行、段后 0 行；

5.截图格式：所有图的标题应在图的下面居中标注。